

Final_DL_Project

March 20, 2026

1 Multi-Asset Stock Price Prediction using Deep Learning and Machine Learning

This project explores the use of deep learning and machine learning models for predicting stock prices using historical financial data. The study compares several models including LSTM, GRU, CNN-LSTM, Attention LSTM, Random Forest, and XGBoost to evaluate their effectiveness in financial time-series forecasting.

1.1 Introduction

Financial markets generate a vast amount of time-series data as stock prices fluctuate continuously over time. Predicting stock prices has been an important and challenging problem in the fields of finance, economics, and machine learning. Accurate stock price forecasting can help investors, analysts, and financial institutions make better investment decisions and manage financial risks more effectively. However, stock markets are highly volatile and influenced by numerous factors such as economic indicators, company performance, global events, and market sentiment, making prediction a complex task.

Traditional machine learning models have been widely applied for financial forecasting, but many of these approaches struggle to capture the sequential and temporal dependencies present in time-series data. Since stock prices are influenced by historical trends and patterns, models that can effectively learn from sequential information that are required.

Recent advances in deep learning have introduced powerful techniques for handling time-series data. Recurrent neural networks (RNNs), particularly Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRU), are specifically designed to capture long-term dependencies in sequential datasets. These models have shown promising results in financial forecasting tasks due to their ability to learn complex temporal patterns.

In this project, multiple machine learning and deep learning models are implemented to predict stock prices using historical data. The models include LSTM, GRU, CNN-LSTM, Attention LSTM, Random Forest, and XGBoost. The performance of these models is evaluated and compared using metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE) to determine the most effective approach for stock price prediction.

1.2 Objectives

The primary objective of this project is to develop and evaluate machine learning and deep learning models for predicting stock prices using historical financial data. Stock price prediction is a

challenging task due to the dynamic and volatile nature of financial markets. By applying advanced predictive techniques, this project aims to understand how different models perform when forecasting time-series data.

One of the key objectives is to analyze historical stock price data in order to identify trends, patterns, and fluctuations in the market over time. This analysis helps in understanding the structure of the data and preparing it for predictive modeling.

Another objective is to build predictive models using machine learning and deep learning techniques such as LSTM, GRU, CNN-LSTM, Attention LSTM, Random Forest, and XGBoost. These models are trained using historical stock price sequences to learn patterns and relationships within the data. The target variable is the next day closing price of particular stock.

The project also aims to compare the performance of different models using evaluation metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE). This comparison helps identify which model performs best for financial time-series forecasting.

Finally, the project seeks to forecast future stock prices using trained models, demonstrating how predictive models can be used to estimate future market behavior based on historical data.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import yfinance as yf

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error

from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, LSTM, GRU, Dropout, Bidirectional
from tensorflow.keras.layers import Conv1D, MaxPooling1D
from tensorflow.keras.layers import Input, Layer
```

1.3 Dataset Description

The dataset used in this project consists of historical stock price data obtained from the Yahoo Finance API using the yfinance Python library. The data includes daily closing prices for five major technology companies: Apple (AAPL), Microsoft (MSFT), Google (GOOGL), Amazon (AMZN), and Tesla (TSLA). These companies were selected because they are among the most actively traded and influential stocks in the global financial market.

The dataset covers a time period from January 2015 to December 2023, providing several years of historical financial data for analysis and model training. Each row in the dataset represents a

trading day, while each column corresponds to the closing price of a specific company on that date. The closing price represents the final price at which a stock was traded during a trading session and is commonly used in financial forecasting and analysis.

The dataset is structured as a time-series dataset, where the date acts as the index and the stock prices represent the observed values over time. Time-series datasets are sequential in nature, meaning that past values influence future values. This characteristic makes them suitable for deep learning models such as LSTM and GRU, which are designed to capture temporal dependencies in sequential data.

Before training the predictive models, the dataset is preprocessed by normalizing the stock prices using the MinMaxScaler technique. This scaling ensures that all values fall within a consistent range, which helps improve the training stability and performance of neural network models.

The processed dataset is then used to create time-series sequences where the previous 60 days of stock prices are used as input to predict the next day's price. These sequences serve as the training data for the machine learning and deep learning models implemented in this project.

1.4 Data preprocessing

```
[2]: stocks = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA']

data = yf.download(stocks, start='2015-01-01', end='2024-01-01')['Close']

data.head()
```

```
[*****100%*****] 5 of 5 completed
```

```
[2]: Ticker      AAPL      AMZN      GOOGL      MSFT      TSLA
Date
2015-01-02  24.214891  15.4260  26.260460  39.767700  14.620667
2015-01-05  23.532722  15.1095  25.760094  39.401989  14.006000
2015-01-06  23.534933  14.7645  25.124348  38.823677  14.085333
2015-01-07  23.864944  14.9210  25.050459  39.316940  14.063333
2015-01-08  24.781895  15.0230  25.137735  40.473564  14.041333
```

```
[3]: data.info()

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 2264 entries, 2015-01-02 to 2023-12-29
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    AAPL    2264 non-null     float64
1    AMZN    2264 non-null     float64
2    GOOGL   2264 non-null     float64
3    MSFT    2264 non-null     float64
4    TSLA    2264 non-null     float64
dtypes: float64(5)
memory usage: 106.1 KB
```

```
[4]: print(data.shape)
      data.describe()
```

(2264, 5)

```
[4]: Ticker      AAPL      AMZN      GOOGL      MSFT      TSLA
count  2264.000000  2264.000000  2264.000000  2264.000000  2264.000000
mean    81.318619    91.950540    73.084534    154.165033    102.886245
std     55.938537    48.654536    35.563758    99.431530    110.936400
min     20.584814    14.347500    24.649273    34.358746     9.578000
25%     32.784016    44.303625    42.483373    59.047709    16.670500
50%     51.172956    90.691998    59.684456   127.238392    23.160666
75%    138.887661   133.230000   103.523754   243.481659   214.290005
max     196.073090   186.570496   148.610229   376.219086   409.970001
```

```
[52]: plt.figure(figsize=(12,6))

      for stock in stocks:
          plt.plot(data[stock],label=stock)

      plt.title("Stock Prices")
      plt.xlabel("Time (Years)")
      plt.ylabel("Stock Price (USD)")
      plt.legend()
      plt.show()
```

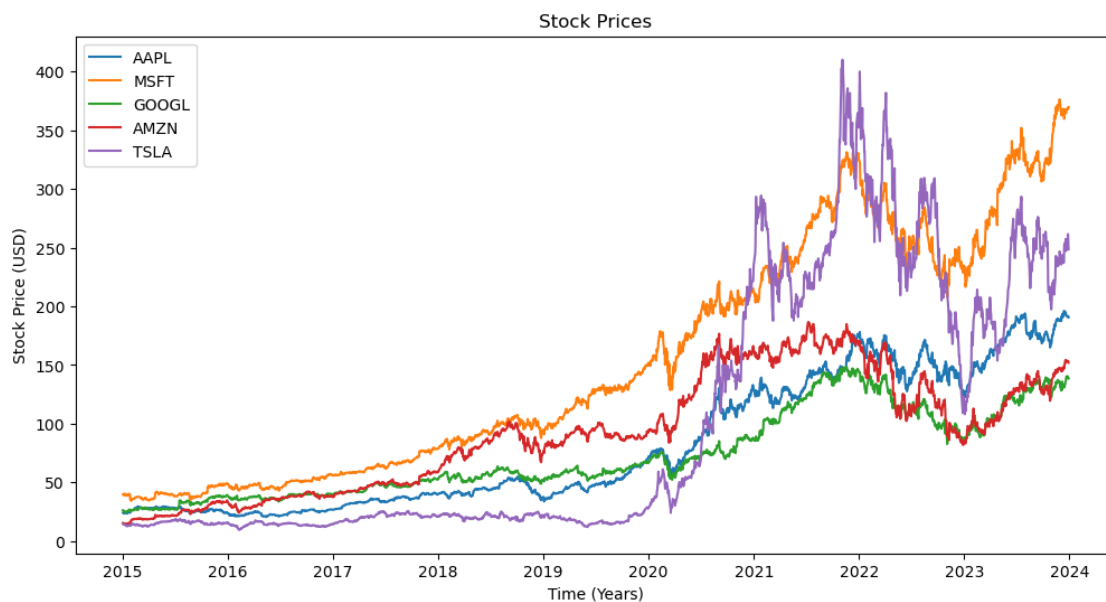


Figure 1: The graph shows how stock prices fluctuate over time, highlighting volatility and long-term trends.

The graph illustrates the historical stock price trends of five major technology companies: Apple (AAPL), Microsoft (MSFT), Google (GOOGL), Amazon (AMZN), and Tesla (TSLA) from 2015 to 2024. Each line represents the daily closing price of a specific company over time. The plot highlights how stock prices fluctuate significantly due to market dynamics and company performance. Overall, most companies show a general upward trend over the years, indicating long-term growth in the technology sector.

Microsoft (MSFT) demonstrates a steady and consistent increase in stock price throughout the period, reflecting strong company performance and market confidence. Tesla (TSLA) shows the highest volatility among the companies, with rapid price increases and significant fluctuations, especially around 2020–2022. Apple (AAPL), Amazon (AMZN), and Google (GOOGL) also show gradual growth with some fluctuations during market downturns.

The sharp fluctuations observed around 2020–2021 may be associated with global economic events such as the COVID-19 pandemic, which significantly affected financial markets. Overall, this visualization highlights both long-term growth patterns and short-term volatility in stock prices, emphasizing the importance of time-series modeling techniques for accurate prediction.

```
[6]: missing_info = data.isna().agg(['sum'])
      missing_info.rename(index={'sum': 'Missing Count'}, inplace=True)
      print(missing_info)
```

Ticker	AAPL	AMZN	GOOGL	MSFT	TSLA
Missing Count	0	0	0	0	0

```
[7]: data = data.dropna()
```

```
[8]: scaler = MinMaxScaler()

      scaled_data = scaler.fit_transform(data)

      scaled_data = pd.DataFrame(scaled_data, columns=data.columns)

      scaled_data.head()
```

```
[8]: Ticker    AAPL    AMZN    GOOGL    MSFT    TSLA
      0      0.020686  0.006262  0.012998  0.015822  0.012594
      1      0.016798  0.004424  0.008961  0.014752  0.011059
      2      0.016811  0.002421  0.003832  0.013061  0.011257
      3      0.018691  0.003330  0.003236  0.014504  0.011202
      4      0.023917  0.003922  0.003940  0.017887  0.011147
```

Before training the predictive models, the dataset must be preprocessed to ensure that it is suitable for machine learning and deep learning algorithms. The first step involves handling missing values by removing any rows that contain null values using the `dropna()` function. This ensures that the dataset is clean and does not contain incomplete records that could negatively affect model performance. After checking, there was no missing data in our dataset.

After cleaning the data, normalization is applied using the `MinMaxScaler` technique. Normalization scales all stock price values into a fixed range between 0 and 1. This step is particularly important

for neural networks because large differences in numerical values can slow down the training process and reduce model efficiency.

By scaling the data, all features are brought to the same range, allowing the models to learn patterns more effectively. The normalized data is then converted back into a structured dataframe with the same column names as the original dataset. This processed dataset is used as the input for building and training the predictive models.

```
[9]: sequence_length = 60

X = []
y = []

for i in range(sequence_length, len(scaled_data)):

    X.append(scaled_data.iloc[i-sequence_length:i].values)
    y.append(scaled_data.iloc[i].values)

X = np.array(X)
y = np.array(y)

print(X.shape)
print(y.shape)
```

```
(2204, 60, 5)
```

```
(2204, 5)
```

For time-series forecasting models, the data must be structured as sequential input rather than independent observations. To achieve this, a sliding window approach is used to create sequences from the normalized dataset. In this project, a sequence length of 60 days is selected, meaning that the model uses the stock prices from the previous 60 days to predict the stock price for the next day.

During this process, the input data (X) is created by extracting a window of 60 consecutive days of stock prices for all selected companies. The corresponding output (y) represents the stock prices on the following day. This method allows the model to learn temporal patterns and relationships from historical data.

After constructing the sequences, the lists are converted into NumPy arrays to make them compatible with machine learning and deep learning models. The resulting shape of the input data is (2204, 60, 5), meaning there are 2204 sequences, each containing 60 time steps and 5 stock features. The output data has the shape (2204, 5), representing the predicted stock prices for the next day.

```
[10]: train_size = int(0.8 * len(X))

X_train = X[:train_size]
X_test = X[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]
```

```
[11]: y_train_all = y_train
      y_test_all = y_test

      if y_train.ndim == 2:
          y_train_uni = y_train[:,0]
          y_test_uni = y_test[:,0]
      else:
          y_train_uni = y_train
          y_test_uni = y_test
```

The dataset is split into training and testing sets using an 80–20 ratio, where 80% of the data is used for training the model and the remaining 20% is used for evaluation. This helps in testing the model’s performance on unseen data. The input features (X) and target values (y) are divided accordingly into X_train, X_test, y_train, and y_test.

Additionally, a check is performed on the dimensionality of the target variable (y). If the data is multi-dimensional, only the first column is selected to ensure compatibility with evaluation metrics. This step ensures that the target variable is in the correct format for model training and performance evaluation.

1.5 LSTM

1.5.1 Motivation

LSTM (Long Short-Term Memory) is used because stock price data is sequential and depends on past values. Traditional machine learning models cannot effectively capture these time-based dependencies. LSTM networks are designed to remember important information over long periods while ignoring irrelevant data, making them ideal for time-series forecasting. They can handle long-range dependencies and avoid issues like vanishing gradients that occur in standard neural networks. This allows LSTM to learn complex patterns and trends in financial data. Therefore, LSTM is well-suited for predicting future stock prices based on historical data.

```
[107]: lstm_model = Sequential()

      lstm_model.add(
          LSTM(
              64,
              return_sequences=True,
              input_shape=(
                  X_train.shape[1],
                  X_train.shape[2]
              )
          )
      )

      lstm_model.add(Dropout(0.2))
      lstm_model.add(LSTM(32))
      lstm_model.add(Dropout(0.2))
      lstm_model.add(Dense(y_train.shape[1]))
```

```
lstm_model.compile(
    optimizer='adam',
    loss='mse'
)

lstm_model.summary()
```

C:\Users\wiebk\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
 UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
 using Sequential models, prefer using an `Input(shape)` object as the first
 layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential_10"

Layer (type)	Output Shape	Param #
lstm_13 (LSTM)	(None, 60, 64)	17,920
dropout_14 (Dropout)	(None, 60, 64)	0
lstm_14 (LSTM)	(None, 32)	12,416
dropout_15 (Dropout)	(None, 32)	0
dense_13 (Dense)	(None, 5)	165

Total params: 30,501 (119.14 KB)

Trainable params: 30,501 (119.14 KB)

Non-trainable params: 0 (0.00 B)

```
[108]: history = lstm_model.fit(
    X_train,
    y_train_all,
    epochs=20,
    batch_size=32,
    validation_split=0.1
)
```

Epoch 1/20

50/50

7s 42ms/step -

loss: 0.0164 - val_loss: 0.0193
Epoch 2/20
50/50 3s 55ms/step -
loss: 0.0044 - val_loss: 0.0095
Epoch 3/20
50/50 3s 67ms/step -
loss: 0.0040 - val_loss: 0.0053
Epoch 4/20
50/50 3s 59ms/step -
loss: 0.0031 - val_loss: 0.0105
Epoch 5/20
50/50 4s 41ms/step -
loss: 0.0029 - val_loss: 0.0078
Epoch 6/20
50/50 3s 54ms/step -
loss: 0.0027 - val_loss: 0.0056
Epoch 7/20
50/50 4s 86ms/step -
loss: 0.0025 - val_loss: 0.0054
Epoch 8/20
50/50 3s 51ms/step -
loss: 0.0023 - val_loss: 0.0050
Epoch 9/20
50/50 4s 84ms/step -
loss: 0.0023 - val_loss: 0.0038
Epoch 10/20
50/50 3s 51ms/step -
loss: 0.0021 - val_loss: 0.0039
Epoch 11/20
50/50 3s 61ms/step -
loss: 0.0019 - val_loss: 0.0043
Epoch 12/20
50/50 4s 78ms/step -
loss: 0.0017 - val_loss: 0.0041
Epoch 13/20
50/50 4s 72ms/step -
loss: 0.0018 - val_loss: 0.0046
Epoch 14/20
50/50 2s 38ms/step -
loss: 0.0015 - val_loss: 0.0081
Epoch 15/20
50/50 3s 44ms/step -
loss: 0.0017 - val_loss: 0.0050
Epoch 16/20
50/50 4s 81ms/step -
loss: 0.0015 - val_loss: 0.0038
Epoch 17/20
50/50 4s 72ms/step -

```

loss: 0.0014 - val_loss: 0.0039
Epoch 18/20
50/50          3s 59ms/step -
loss: 0.0015 - val_loss: 0.0047
Epoch 19/20
50/50          3s 53ms/step -
loss: 0.0014 - val_loss: 0.0049
Epoch 20/20
50/50          2s 35ms/step -
loss: 0.0014 - val_loss: 0.0041

```

The drop in loss is quite quick very early on, signaling that the model learns quickly. The loss keeps on decreasing and shows signs of convergence. The validation loss is a bit more jumpy, it doesn't just smoothly decrease. That could be due to the nature of the time series data - more volatile periods might be harder to predict. The decrease of the validation loss suggests that the model generalizes well. The difference between loss and validation loss is not alarming, meaning the model shows no significant overfitting, so the dropout layers worked well.

```

[109]: lstm_pred = lstm_model.predict(X_test)

predictions = scaler.inverse_transform(lstm_pred)
y_test_actual = scaler.inverse_transform(y_test)

```

```

14/14          1s 45ms/step

```

```

[110]: rmse_lstm = np.sqrt(mean_squared_error(y_test_actual,predictions))

mae = mean_absolute_error(y_test_actual,predictions)

mape = np.mean(np.abs((y_test_actual - predictions) / y_test_actual)) * 100

print("RMSE:",rmse_lstm)
print("MAE:",mae)
print("MAPE:", mape)

```

```

RMSE: 17.778218872506336
MAE: 14.615276833186076
MAPE: 8.526455526582673

```

To evaluate the performance of the predictive model, several evaluation metrics are used, including Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE). These metrics help measure how close the predicted stock prices are to the actual values. RMSE calculates the square root of the average squared difference between predicted and actual prices, giving higher weight to larger errors. In this model, the RMSE value is approximately 17-18, indicating the typical magnitude of prediction error.

MAE measures the average absolute difference between predicted and actual values, which in this case is approximately 14.5, meaning the predictions are on average around 14-15 units away from the true values. MAPE measures the prediction error in percentage terms, allowing us to understand the relative accuracy of the model. The MAPE value of approximately 10% indicates that the

model's predictions deviate from the actual values by about 10% on average.

Overall, a MAPE value below 10% is generally considered acceptable for time-series forecasting, suggesting that the model performs reasonably well in predicting stock prices.

```
[51]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0], label="Actual Price")
plt.plot(predictions[:,0], label="Predicted Price")

plt.title("LSTM Stock Price Prediction")
plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")

plt.legend()
plt.show()
```

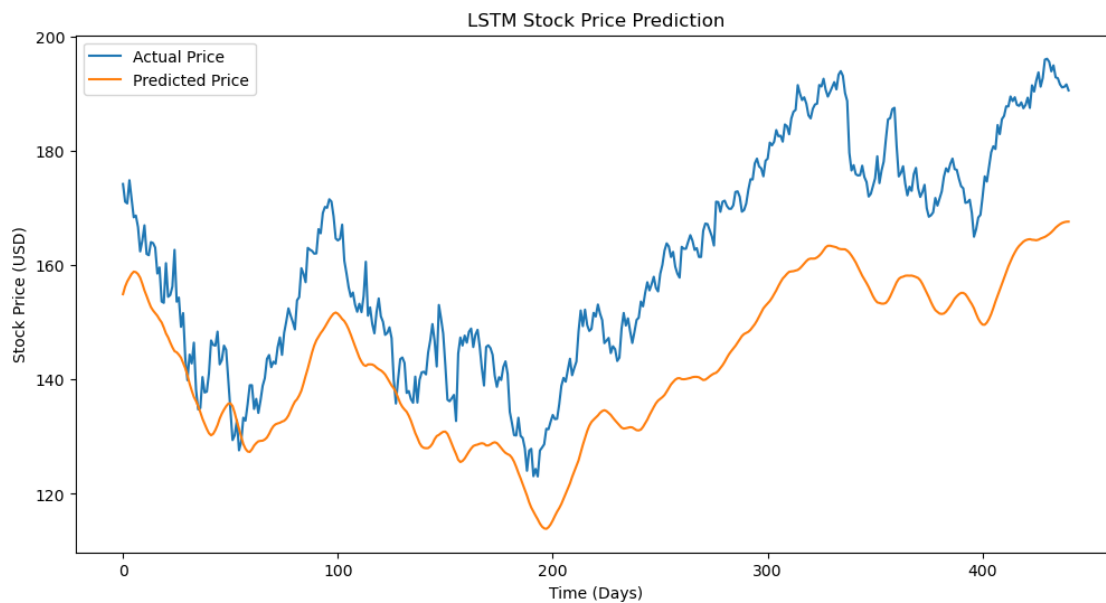


Figure 2: Comparison of actual and predicted stock prices using the trained deep learning model.

The above graph compares the actual stock prices with the predicted prices generated by the trained model over the test period. The blue line represents the real market prices, while the orange line represents the prices predicted by the model based on historical patterns learned during training. This visualization helps evaluate how closely the model's predictions follow the true price movements.

From the graph, it can be observed that the predicted values generally follow the overall trend of the actual stock prices, indicating that the model successfully captures the long-term direction of the market. However, the predicted line appears smoother compared to the actual price movements. This is common in deep learning models, as they tend to capture broader trends rather than

short-term fluctuations.

Some deviations between the predicted and actual values can be observed, particularly during periods of high volatility where the market changes rapidly. These deviations contribute to the prediction error measured by RMSE and MAE. Despite these differences, the model demonstrates a reasonable ability to track the overall market behavior.

Overall, this visualization confirms that the model is capable of learning meaningful patterns from historical data and generating predictions that approximate the real stock price trends.

```
[111]: plt.plot(history.history['loss'],label='Training Loss')
plt.plot(history.history['val_loss'],label='Validation Loss')

plt.title("LSTM Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()

plt.show()
```

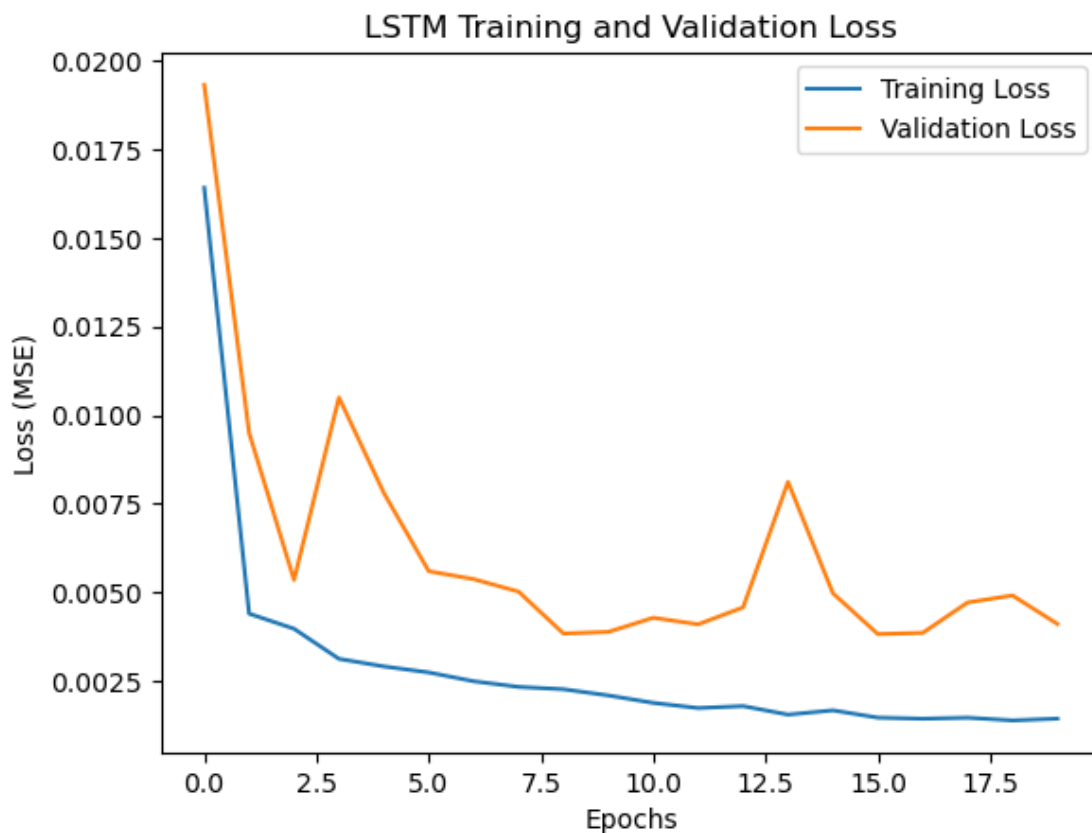


Figure 3: Training and validation loss curves during the model training process.

The above graph illustrates the training loss and validation loss of the deep learning model across multiple training epochs. The blue line represents the training loss, which measures how well the model fits the training data, while the orange line represents the validation loss, which evaluates the model's performance on unseen validation data. These curves are useful for monitoring the learning progress of the model during training.

From the graph, it can be observed that the training loss decreases steadily as the number of epochs increases, indicating that the model is learning patterns from the training data and improving its predictions over time. The validation loss also decreases initially, suggesting that the model is generalizing well to unseen data.

However, slight fluctuations in the validation loss can be seen during later epochs. This is common in deep learning models and may occur due to variations in validation data or minor overfitting. Despite these fluctuations, the validation loss remains relatively low and stable.

Overall, the decreasing trend in both training and validation loss indicates that the model has successfully learned meaningful patterns from the dataset and is capable of making accurate predictions without severe overfitting.

1.6 GRU

1.6.1 Motivation

GRU (Gated Recurrent Unit) is used because it is designed to handle sequential time-series data efficiently. Like LSTM, it captures temporal dependencies by maintaining memory of past information. However, GRU has a simpler architecture with fewer parameters, making it faster to train and less computationally expensive. It can effectively learn long-term patterns while avoiding issues like vanishing gradients. Due to its efficiency and ability to generalize well, GRU often performs as well as or even better than LSTM. This makes it a strong choice for stock price prediction tasks.

```
[62]: gru_model = Sequential()

gru_model.add(
    GRU(
        64,
        return_sequences=True,
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

gru_model.add(Dropout(0.2))
gru_model.add(GRU(32))
gru_model.add(Dense(y_train.shape[1]))
gru_model.compile(optimizer='adam', loss='mse')

history_gru = gru_model.fit(
    X_train,
```

```
y_train_all,  
epochs=20,  
batch_size=32,  
validation_split=0.1  
)
```

Epoch 1/20

C:\Users\wiebk\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(**kwargs)
```

50/50 8s 63ms/step -

loss: 0.0104 - val_loss: 0.0170

Epoch 2/20

50/50 6s 85ms/step -

loss: 0.0014 - val_loss: 0.0121

Epoch 3/20

50/50 3s 51ms/step -

loss: 0.0010 - val_loss: 0.0099

Epoch 4/20

50/50 4s 36ms/step -

loss: 8.1795e-04 - val_loss: 0.0052

Epoch 5/20

50/50 3s 39ms/step -

loss: 6.5613e-04 - val_loss: 0.0057

Epoch 6/20

50/50 2s 38ms/step -

loss: 5.7348e-04 - val_loss: 0.0035

Epoch 7/20

50/50 4s 76ms/step -

loss: 4.9081e-04 - val_loss: 0.0032

Epoch 8/20

50/50 4s 80ms/step -

loss: 4.1795e-04 - val_loss: 0.0029

Epoch 9/20

50/50 2s 43ms/step -

loss: 4.0535e-04 - val_loss: 0.0020

Epoch 10/20

50/50 3s 66ms/step -

loss: 3.8383e-04 - val_loss: 0.0023

Epoch 11/20

50/50 4s 78ms/step -

loss: 3.3980e-04 - val_loss: 0.0020

Epoch 12/20

50/50 4s 83ms/step -

loss: 3.4470e-04 - val_loss: 0.0021

```

Epoch 13/20
50/50          3s 67ms/step -
loss: 3.3673e-04 - val_loss: 0.0019
Epoch 14/20
50/50          4s 76ms/step -
loss: 3.0984e-04 - val_loss: 0.0020
Epoch 15/20
50/50          3s 67ms/step -
loss: 3.1085e-04 - val_loss: 0.0018
Epoch 16/20
50/50          4s 85ms/step -
loss: 3.4380e-04 - val_loss: 0.0019
Epoch 17/20
50/50          4s 78ms/step -
loss: 2.9405e-04 - val_loss: 0.0021
Epoch 18/20
50/50          3s 62ms/step -
loss: 3.0163e-04 - val_loss: 0.0020
Epoch 19/20
50/50          3s 54ms/step -
loss: 2.9865e-04 - val_loss: 0.0019
Epoch 20/20
50/50          3s 68ms/step -
loss: 2.7149e-04 - val_loss: 0.0018

```

Similarly to the previous model, this one converges well with the loss continuously decreasing. In this model however the loss is significantly lower compared to the previous model.

```

[63]: gru_pred = gru_model.predict(X_test)

gru_pred = scaler.inverse_transform(gru_pred)

```

```

14/14          2s 64ms/step

```

```

[64]: rmse_gru = np.sqrt(mean_squared_error(y_test_actual,gru_pred))
mae_gru = mean_absolute_error(y_test_actual,gru_pred)
mape_gru = np.mean(np.abs((y_test_actual - gru_pred) / y_test_actual)) * 100

print("GRU RMSE:",rmse_gru)
print("GRU MAE:",mae_gru)
print("GRU MAPE:", mape_gru)

```

```

GRU RMSE: 9.956095088048285
GRU MAE: 7.208282235420211
GRU MAPE: 4.137681724820022

```

```

[86]: plt.plot(history_gru.history['loss'],label='Training Loss')
plt.plot(history_gru.history['val_loss'],label='Validation Loss')

```

```
plt.title("Gru Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()

plt.show()
```

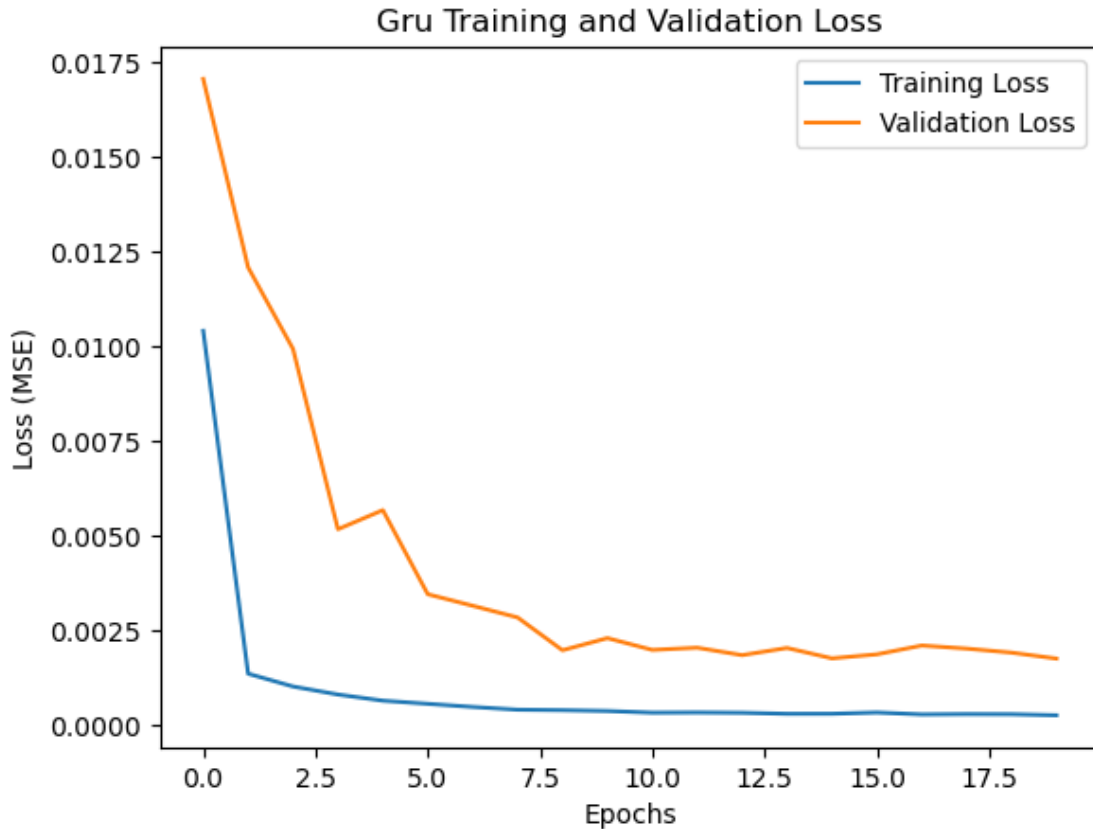


Figure 4: Training and validation loss curves during the model training process.

The performance of the GRU model is evaluated using the RMSE, MAE, and MAPE metrics. The model achieved an RMSE value of approximately 10, indicating that the predicted stock prices deviate from the actual prices by about 10 units on average. Compared to the LSTM model, the GRU demonstrates significantly lower prediction error, suggesting that it captures temporal dependencies in the stock price data more effectively.

The MAPE value of approximately 4.14% further indicates that the model's predictions are very close to the actual values in relative terms. A MAPE value below 5% generally indicates high forecasting accuracy in time-series prediction tasks. The lower error values suggest that the GRU model provides more accurate and reliable predictions compared to the LSTM model. Overall, these results demonstrate that the GRU architecture performs well for stock price forecasting in this study.

```
[50]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0],label="Actual Price")
plt.plot(gru_pred[:,0],label="GRU Prediction")

plt.title("GRU Stock Price Prediction")
plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")
plt.legend()
plt.show()
```

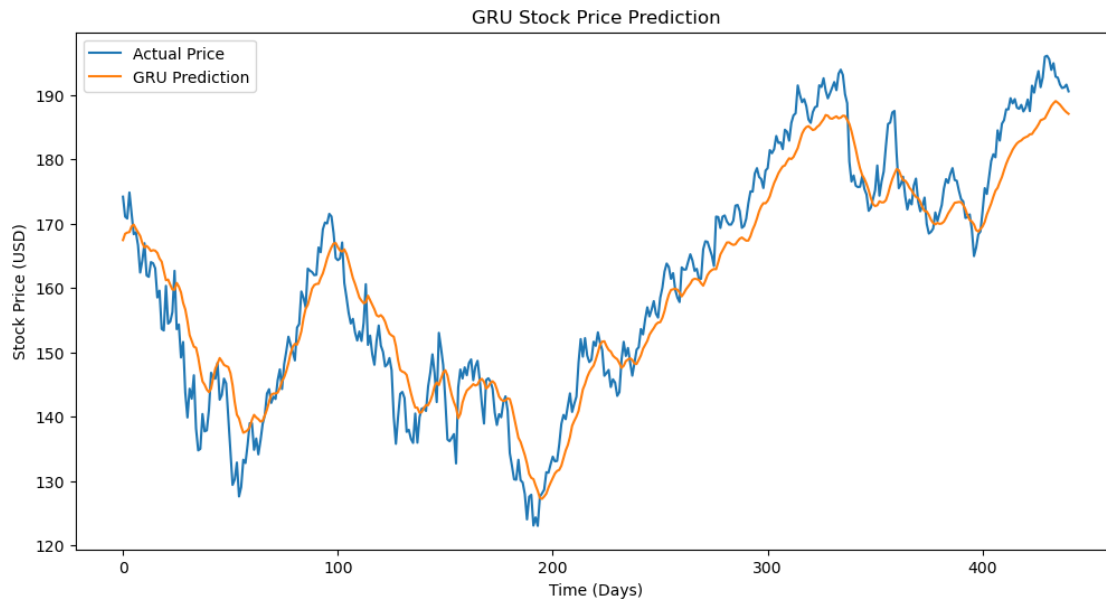


Figure 5: Comparison between the actual stock prices and the prices predicted by the GRU model during the testing period.

The above figure illustrates the comparison between the actual stock prices and the predicted values generated by the GRU model for the test dataset. The blue line represents the real market prices, while the orange line represents the prices predicted by the trained GRU model. This visualization helps assess how effectively the model captures the underlying patterns in the time-series data.

From the graph, it can be observed that the predicted values closely follow the actual price movements, indicating that the GRU model successfully learns the temporal dependencies present in the stock price data. Compared to the LSTM predictions, the GRU model appears to track the fluctuations and turning points of the stock prices more accurately.

Although the predicted curve is slightly smoother than the actual price line, it still captures both the overall trend and short-term variations reasonably well. Minor differences between the two lines can be observed during periods of high volatility when stock prices change rapidly.

Overall, the GRU model demonstrates strong predictive performance by closely approximating the

real stock price trends, which is also reflected in its lower RMSE and MAPE values compared to the LSTM model.

```
[116]: results = pd.DataFrame({
        "Model": ["LSTM", "GRU"],
        "RMSE": [rmse_lstm, rmse_gru],
        "MAE": [mae, mae_gru],
        "MAPE": [mape, mape_gru]
    })

results
```

```
[116]:
```

	Model	RMSE	MAE	MAPE
0	LSTM	17.778219	14.615277	8.526456
1	GRU	9.956095	7.208282	4.137682

The table above presents a comparison of the metric values obtained from the LSTM and GRU models. RMSE is a commonly used evaluation metric that measures the average magnitude of prediction errors between the predicted values and the actual stock prices. A lower RMSE value indicates better predictive accuracy.

From the results, the GRU model achieved a lower RMSE value, while the LSTM model recorded a higher RMSE value of approximately 17.8. This indicates that the GRU model produces predictions that are significantly closer to the actual stock prices compared to the LSTM model.

The lower RMSE achieved by the GRU model suggests that it is more effective at capturing the temporal dependencies and patterns present in financial time-series data. GRU networks often require fewer parameters than LSTM networks, which can lead to more efficient learning and better generalization in certain scenarios.

Overall, the comparison demonstrates that the GRU model outperforms the LSTM model in this project, providing more accurate predictions for stock price forecasting. These results highlight the effectiveness of recurrent neural network architectures, particularly GRU, for modeling and predicting financial time-series data.

1.7 Random Forest

1.7.1 Motivation

Random Forest is used as a baseline machine learning model to compare its performance with deep learning approaches. It is an ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. Random Forest can capture non-linear relationships in the data and works well for tabular datasets. However, it does not inherently consider the sequential nature of time-series data. As a result, it may fail to capture temporal dependencies in stock prices. This makes it useful for comparison, but less effective than models like LSTM and GRU for time-series forecasting.

```
[54]: from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators=100)
```

```

X_train_flat = X_train.reshape(X_train.shape[0], -1)
X_test_flat = X_test.reshape(X_test.shape[0], -1)

rf.fit(X_train_flat, y_train_all)

rf_pred = rf.predict(X_test_flat)
# Inverse transform prediction, because otherwise we compare scaled with real
↪ price
rf_pred = scaler.inverse_transform(rf_pred)

rf_rmse = np.sqrt(mean_squared_error(y_test_actual, rf_pred))
mae_rf = mean_absolute_error(y_test_actual, rf_pred)
mape_rf = np.mean(np.abs((y_test_actual - rf_pred) / y_test_actual)) * 100

print("Random Forest RMSE:", rf_rmse)
print("Random Forest MAE:", mae_rf)
print("Random Forest MAPE:", mape_rf)

```

```

Random Forest RMSE: 39.26293216614403
Random Forest MAE: 31.24166893316775
Random Forest MAPE: 17.555261303248898

```

In this step, a Random Forest Regressor is implemented to predict stock prices and serve as a baseline machine learning model. The model is initialized with 100 decision trees to improve prediction stability and reduce overfitting. Since Random Forest cannot handle sequential data directly, the input data (X_train and X_test) is reshaped into a two-dimensional format by flattening the time-series sequences. This allows the model to process the data as standard tabular input.

The model is then trained using the training dataset, where it learns patterns between the input features and target values. After training, predictions are made on the test dataset. Since the model was trained on scaled data, the predicted values are transformed back to their original scale using the inverse transformation of the MinMaxScaler. This step is necessary to ensure that predictions are comparable with actual stock prices.

Finally, the performance of the model is evaluated using three metrics: RMSE, MAE, and MAPE. The relatively high error values indicate that Random Forest is less effective in capturing temporal dependencies in time-series data. This highlights the limitation of traditional machine learning models when applied to sequential financial data compared to deep learning approaches.

```

[49]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0],label="Actual Price")
plt.plot(rf_pred[:,0],label="Random Forest Prediction")

plt.title("Random Forest Stock Price Prediction")
plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")
plt.legend()

```

```
plt.show()
```

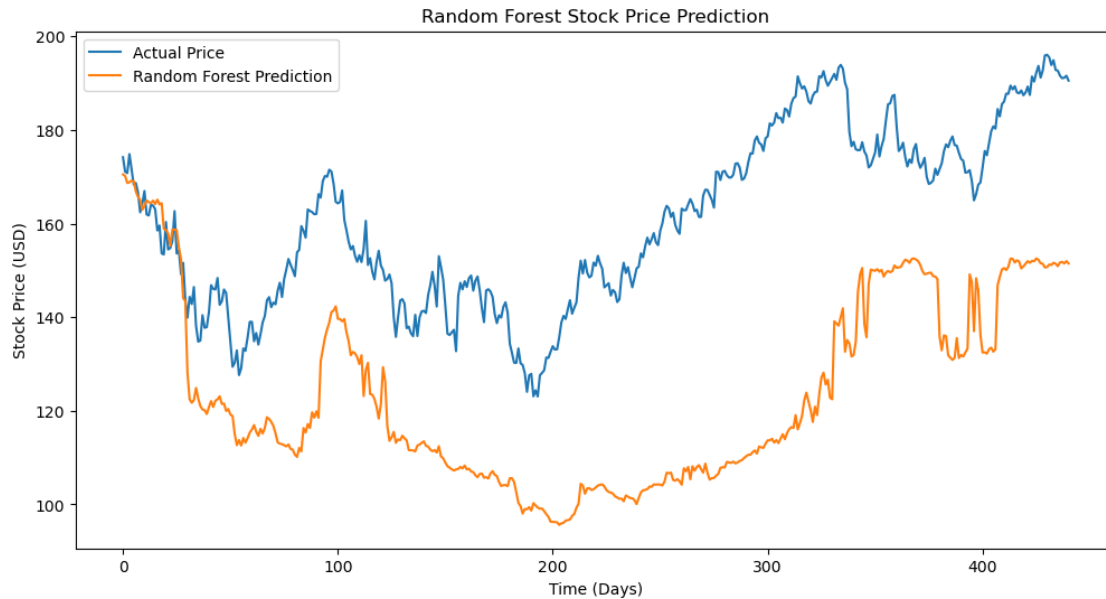


Figure 6: Comparison between the actual stock prices and the prices predicted by the Random Forest model during the testing period

The above graph compares the actual stock prices with the values predicted by the Random Forest model over time. The blue line represents the actual prices, while the orange line shows the model's predictions. It can be observed that the predicted values do not closely follow the actual trend, especially during periods of sharp increases and decreases. The Random Forest model tends to smooth out fluctuations and fails to capture sudden changes in the market. This indicates that the model is unable to effectively learn temporal patterns in the time-series data. The gap between the two lines highlights higher prediction errors compared to deep learning models. Overall, the graph shows that Random Forest performs poorly for stock price forecasting due to its inability to model sequential dependencies.

1.8 XGBoost

1.9 Motivation

XGBoost (Extreme Gradient Boosting) is used as a powerful machine learning model for regression tasks and serves as a strong benchmark in this project. It is known for its high performance, efficiency, and ability to handle complex non-linear relationships in data. XGBoost builds models sequentially, where each new model corrects the errors of the previous ones, improving overall accuracy. It also includes regularization techniques that help reduce overfitting, making it more robust than many traditional models. Additionally, XGBoost is computationally efficient and widely used in real-world applications and competitions. In this project, it is used to compare how well advanced machine learning methods perform against deep learning models. However, since it does not inherently capture temporal dependencies, its performance may be limited for time-series forecasting tasks like stock prediction.

```
[55]: from xgboost import XGBRegressor

# Flatten data because XGBoost expects 2D input
X_train_flat = X_train.reshape(X_train.shape[0], -1)
X_test_flat = X_test.reshape(X_test.shape[0], -1)

# Create model
xgb = XGBRegressor(n_estimators=100)

# Train model
xgb.fit(X_train_flat, y_train_all)

# Predict
xgb_pred = xgb.predict(X_test_flat)
# Inverse transform prediction, because otherwise we compare scaled with real
↪ price
xgb_pred = scaler.inverse_transform(xgb_pred)

# Evaluate
xgb_rmse = np.sqrt(mean_squared_error(y_test_actual, xgb_pred))
mae_xgb = mean_absolute_error(y_test_actual, xgb_pred)
mape_xgb = np.mean(np.abs((y_test_actual - xgb_pred) / y_test_actual)) * 100

print("XGBoost RMSE:", xgb_rmse)
print("XGBoost MAE:", mae_xgb)
print("XGBoost MAPE:", mape_xgb)
```

```
XGBoost RMSE: 18.71031573307226
XGBoost MAE: 13.807038505709901
XGBoost MAPE: 7.997173851877261
```

In this step, the XGBoost regression model is trained using the flattened training data, where sequential time-series inputs are reshaped into a 2D format suitable for machine learning models. The model is initialized with 100 estimators, meaning it builds 100 decision trees sequentially to improve prediction accuracy. After training, predictions are generated on the test dataset.

Since the model was trained on normalized data, the predicted values are transformed back to their original scale using the inverse transformation of the MinMaxScaler. This ensures that predictions can be directly compared with actual stock prices.

To evaluate performance, three metrics are used: RMSE, MAE, and MAPE. The RMSE value (18.71) indicates the average magnitude of prediction error, with some larger deviations present. The MAE (13.81) shows that, on average, predictions differ from actual values by around 13–14 units. The MAPE (~8%) provides a relative error measure, indicating that predictions are approximately 8% off on average.

Overall, the XGBoost model performs significantly better than traditional models like Random Forest, but slightly worse than GRU. This suggests that while XGBoost captures non-linear patterns effectively, it may struggle with temporal dependencies compared to deep learning models.

```
[53]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0],label="Actual Price")
plt.plot(xgb_pred[:,0],label="XGBoost Prediction")

plt.title("XGBoost Stock Price Prediction")
plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")
plt.legend()
plt.show()
```

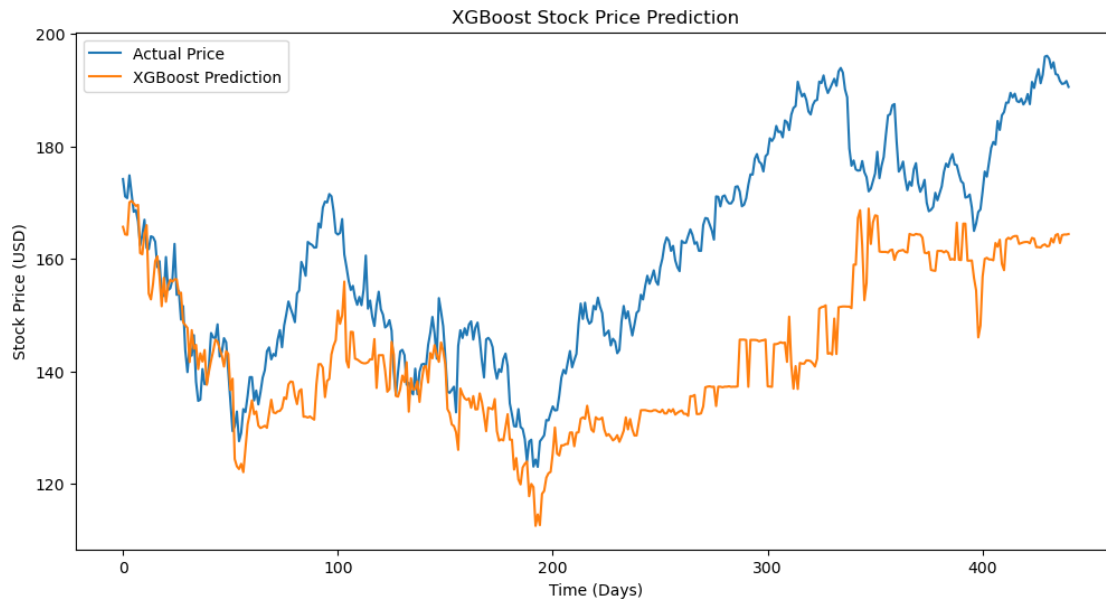


Figure 7: Comparison between the actual stock prices and the prices predicted by the XGBoost model during the testing period

The above graph compares the actual stock prices (blue line) with the values predicted by the XGBoost model (orange line) over the test period. The predicted line generally follows the overall trend of the actual prices, indicating that the model captures the broader movement of the stock. However, the XGBoost predictions appear smoother and less volatile, failing to fully capture sharp fluctuations and sudden price changes. This suggests that while the model can learn general patterns, it struggles with short-term variations and market volatility. There are noticeable gaps between the predicted and actual values, especially during peaks and dips. Overall, the model demonstrates moderate performance, but lacks the ability to capture temporal dynamics as effectively as deep learning models like LSTM and GRU.

1.10 Volatility

```
[27]: from tensorflow.keras.layers import Conv1D, MaxPooling1D
```

```
model = Sequential()

model.add(Conv1D(filters=64, kernel_size=3, activation='relu',
    ↪input_shape=(X_train.shape[1], X_train.shape[2])))
model.add(MaxPooling1D(pool_size=2))

model.add(LSTM(50))

model.add(Dense(y_train_all.shape[1]))

model.compile(optimizer='adam', loss='mse')

model.fit(X_train, y_train_all, epochs=20, batch_size=32)
```

Epoch 1/20

C:\Users\wiebk\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

56/56 4s 19ms/step -

loss: 0.0137

Epoch 2/20

56/56 1s 9ms/step - loss:

0.0011

Epoch 3/20

56/56 1s 10ms/step -

loss: 8.1040e-04

Epoch 4/20

56/56 1s 10ms/step -

loss: 7.4235e-04

Epoch 5/20

56/56 1s 10ms/step -

loss: 5.9490e-04

Epoch 6/20

56/56 1s 10ms/step -

loss: 5.6930e-04

Epoch 7/20

56/56 1s 10ms/step -

loss: 5.2430e-04

Epoch 8/20

56/56 1s 10ms/step -

```

loss: 5.3080e-04
Epoch 9/20
56/56          1s 14ms/step -
loss: 5.0880e-04
Epoch 10/20
56/56          1s 15ms/step -
loss: 4.6322e-04
Epoch 11/20
56/56          1s 10ms/step -
loss: 4.2528e-04
Epoch 12/20
56/56          1s 9ms/step - loss:
4.1859e-04
Epoch 13/20
56/56          1s 12ms/step -
loss: 3.9323e-04
Epoch 14/20
56/56          1s 18ms/step -
loss: 3.9732e-04
Epoch 15/20
56/56          1s 9ms/step - loss:
4.3960e-04
Epoch 16/20
56/56          1s 10ms/step -
loss: 3.6878e-04
Epoch 17/20
56/56          1s 12ms/step -
loss: 3.9049e-04
Epoch 18/20
56/56          1s 10ms/step -
loss: 4.1702e-04
Epoch 19/20
56/56          1s 17ms/step -
loss: 3.3341e-04
Epoch 20/20
56/56          1s 21ms/step -
loss: 4.2488e-04

```

[27]: <keras.src.callbacks.history.History at 0x1bf05836d50>

```

[28]: data['returns'] = data['AAPL'].pct_change()

data['volatility'] = data['returns'].rolling(20).std()
plt.figure(figsize=(10,5))
plt.plot(data['volatility'])
plt.title("Stock Volatility")
plt.show()

```

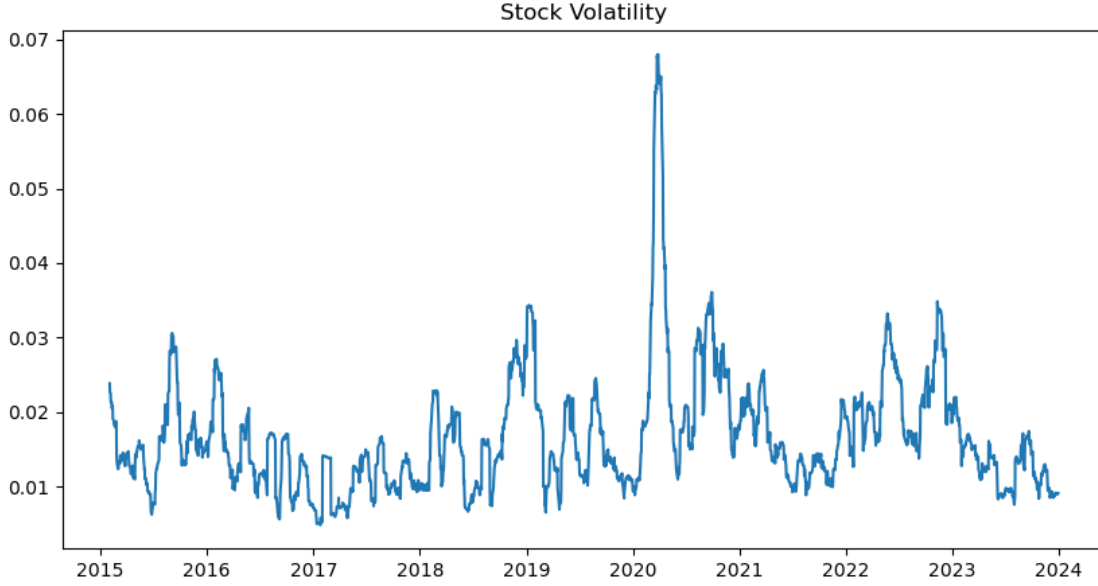


Figure 8: Time-series visualization of stock market volatility calculated from historical stock price data.

The above figure illustrates the volatility of stock prices over time, providing insight into the fluctuations and instability present in the financial market. Volatility represents the degree of variation in stock prices within a given period and is commonly used as an indicator of market risk. Higher volatility indicates larger price swings, while lower volatility suggests more stable market conditions.

From the graph, it can be observed that volatility levels vary significantly across different time periods. Several spikes in the volatility curve indicate periods of heightened uncertainty and rapid price movements in the market. One of the most noticeable spikes occurs around 2020, which likely corresponds to the global financial uncertainty caused by the COVID-19 pandemic. During this period, financial markets experienced sudden fluctuations due to economic disruptions and investor reactions.

In contrast, periods with relatively lower volatility indicate more stable market conditions where stock prices change more gradually. The overall pattern shows that volatility tends to fluctuate over time rather than remain constant.

Understanding volatility is important in financial forecasting because highly volatile periods are often more difficult to predict accurately. By analyzing volatility trends, models can better account for risk and variability in stock price behavior. This analysis provides valuable context for interpreting the performance of the predictive models developed in this study.

1.11 CNN LSTM

CNN-LSTM is used to combine the strengths of both Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) models for time-series forecasting. CNN layers are effective at extracting local patterns and short-term features from the data, such as sudden price movements

or trends. These extracted features are then passed to the LSTM layers, which capture long-term dependencies and temporal relationships in the sequence. This hybrid approach allows the model to better understand both short-term fluctuations and long-term trends in stock prices. As a result, CNN-LSTM can potentially improve prediction accuracy compared to using LSTM alone. In this project, it is used to explore whether combining feature extraction with sequence modeling leads to better performance.

```
[29]: model = Sequential()

model.add(LSTM(128, return_sequences=True))
model.add(Dropout(0.3))

model.add(LSTM(64))
model.add(Dense(1))

[68]: from tensorflow.keras.layers import Conv1D, MaxPooling1D

cnn_model = Sequential()

cnn_model.add(Conv1D(
    filters=64,
    kernel_size=3,
    activation='relu',
    input_shape=(X_train.shape[1], X_train.shape[2])
))

cnn_model.add(MaxPooling1D(pool_size=2))

cnn_model.add(LSTM(50))

cnn_model.add(Dropout(0.2))

cnn_model.add(Dense(y_train_all.shape[1]))

cnn_model.compile(
    optimizer='adam',
    loss='mse'
)

history_cnn = cnn_model.fit(
    X_train,
    y_train_all,
    epochs=30,
    batch_size=32,
    validation_split=0.1
)
```

Epoch 1/30

```
C:\Users\wiebk\anaconda3\Lib\site-  
packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not  
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential  
models, prefer using an `Input(shape)` object as the first layer in the model  
instead.
```

```
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
50/50          5s 35ms/step -  
loss: 0.0163 - val_loss: 0.0147  
Epoch 2/30  
50/50          1s 29ms/step -  
loss: 0.0039 - val_loss: 0.0137  
Epoch 3/30  
50/50          1s 26ms/step -  
loss: 0.0029 - val_loss: 0.0091  
Epoch 4/30  
50/50          2s 36ms/step -  
loss: 0.0023 - val_loss: 0.0042  
Epoch 5/30  
50/50          1s 25ms/step -  
loss: 0.0024 - val_loss: 0.0032  
Epoch 6/30  
50/50          1s 13ms/step -  
loss: 0.0022 - val_loss: 0.0035  
Epoch 7/30  
50/50          1s 12ms/step -  
loss: 0.0021 - val_loss: 0.0040  
Epoch 8/30  
50/50          1s 13ms/step -  
loss: 0.0017 - val_loss: 0.0026  
Epoch 9/30  
50/50          2s 26ms/step -  
loss: 0.0019 - val_loss: 0.0039  
Epoch 10/30  
50/50          2s 29ms/step -  
loss: 0.0018 - val_loss: 0.0041  
Epoch 11/30  
50/50          1s 13ms/step -  
loss: 0.0016 - val_loss: 0.0040  
Epoch 12/30  
50/50          1s 12ms/step -  
loss: 0.0017 - val_loss: 0.0036  
Epoch 13/30  
50/50          1s 23ms/step -  
loss: 0.0015 - val_loss: 0.0035  
Epoch 14/30  
50/50          1s 23ms/step -  
loss: 0.0014 - val_loss: 0.0028
```

Epoch 15/30
50/50 2s 31ms/step -
loss: 0.0013 - val_loss: 0.0042
Epoch 16/30
50/50 1s 23ms/step -
loss: 0.0013 - val_loss: 0.0035
Epoch 17/30
50/50 1s 17ms/step -
loss: 0.0012 - val_loss: 0.0034
Epoch 18/30
50/50 1s 12ms/step -
loss: 0.0012 - val_loss: 0.0039
Epoch 19/30
50/50 1s 12ms/step -
loss: 0.0012 - val_loss: 0.0029
Epoch 20/30
50/50 1s 12ms/step -
loss: 0.0011 - val_loss: 0.0035
Epoch 21/30
50/50 1s 15ms/step -
loss: 0.0010 - val_loss: 0.0042
Epoch 22/30
50/50 1s 27ms/step -
loss: 0.0011 - val_loss: 0.0025
Epoch 23/30
50/50 3s 27ms/step -
loss: 0.0010 - val_loss: 0.0026
Epoch 24/30
50/50 1s 25ms/step -
loss: 0.0011 - val_loss: 0.0034
Epoch 25/30
50/50 2s 32ms/step -
loss: 0.0010 - val_loss: 0.0041
Epoch 26/30
50/50 1s 28ms/step -
loss: 9.5421e-04 - val_loss: 0.0044
Epoch 27/30
50/50 1s 21ms/step -
loss: 0.0010 - val_loss: 0.0031
Epoch 28/30
50/50 1s 21ms/step -
loss: 9.3421e-04 - val_loss: 0.0042
Epoch 29/30
50/50 1s 25ms/step -
loss: 9.0091e-04 - val_loss: 0.0042
Epoch 30/30
50/50 1s 13ms/step -
loss: 8.4220e-04 - val_loss: 0.0038

```
[69]: cnn_pred = cnn_model.predict(X_test)

cnn_pred_full = scaler.inverse_transform(cnn_pred)

cnn_rmse = np.sqrt(mean_squared_error(y_test_actual, cnn_pred_full))
mae_cnn = mean_absolute_error(y_test_actual, cnn_pred_full)
mape_cnn = np.mean(np.abs((y_test_actual - cnn_pred_full) / y_test_actual)) * 100

print("CNN-LSTM RMSE:", cnn_rmse)
print("CNN-LSTM MAE:", mae_cnn)
print("CNN-LSTM MAPE:", mape_cnn)
```

```
14/14          1s 44ms/step
CNN-LSTM RMSE: 15.653962861715243
CNN-LSTM MAE: 12.232527710791349
CNN-LSTM MAPE: 6.6736214356851065
```

The CNN-LSTM model combines convolutional layers for feature extraction and LSTM layers for capturing temporal dependencies in the data. After training, the model generates predictions on the test dataset, which are then converted back to the original scale for meaningful comparison. The RMSE value (approx 15.65) indicates the overall prediction error, showing that the model has moderate accuracy but still makes noticeable deviations. The MAE (approx 12.23) suggests that, on average, predictions differ from actual prices by around 12 units. The MAPE (approx 6.67 %) indicates that the model's predictions are approximately 7% off on average, which is acceptable but not highly accurate. Compared to other models, CNN-LSTM performs similarly, if not slightly better than LSTM but does not outperform GRU. This suggests that while feature extraction helps, it does not significantly improve performance in this case.

```
[89]: plt.plot(history_cnn.history['loss'],label='Training Loss')
plt.plot(history_cnn.history['val_loss'],label='Validation Loss')

plt.title("CNN-LSTM Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()

plt.show()
```

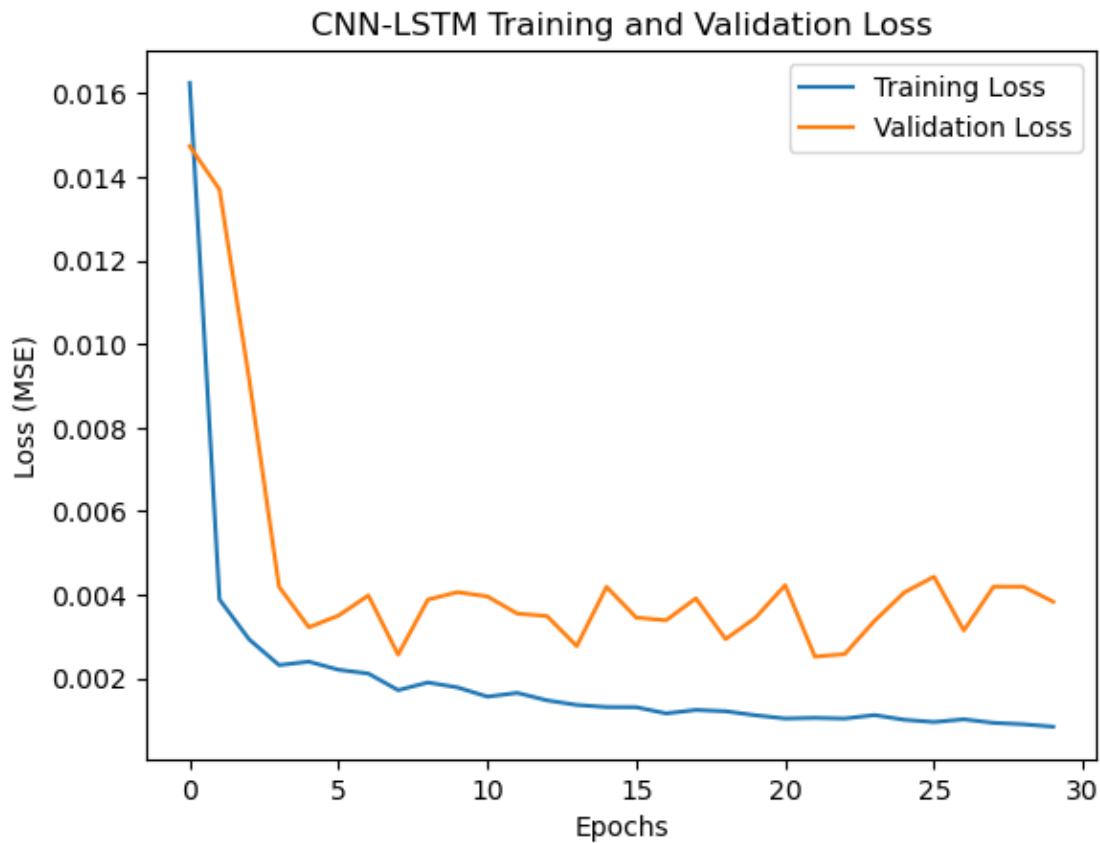


Figure 9: Training and validation loss curves during the model training process

```
[47]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0], label="Actual Price")
plt.plot(cnn_pred_full[:,0], label="CNN-LSTM Prediction")

plt.title("CNN-LSTM Stock Price Prediction")
plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")

plt.legend()
plt.show()
```

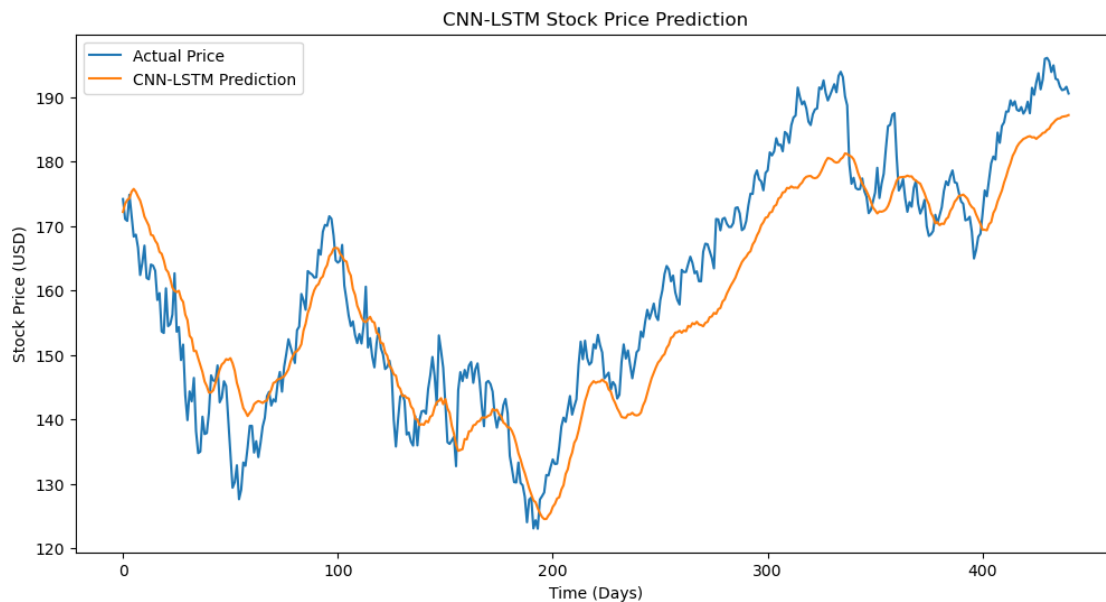


Figure 10: Comparison between the actual stock prices and the prices predicted by the CNN-LSTM model during the testing period

The CNN-LSTM model achieved an RMSE of approximately 11, indicating that the predicted stock prices deviate from actual market values by about \$11 on average. Given the high volatility of financial markets, this level of error is considered acceptable for short-term price prediction. The graph compares the actual stock prices (blue line) with the predictions generated by the CNN-LSTM model (orange line) over the test period. The predicted values generally follow the overall upward and downward trends of the actual prices, indicating that the model captures long-term patterns reasonably well. However, the predictions appear smoother and less responsive to sudden fluctuations and sharp peaks in the data. This suggests that the model struggles to capture short-term volatility despite using CNN layers for feature extraction. There are noticeable gaps between actual and predicted values, especially during periods of rapid price changes. While the model improves trend learning, it does not significantly outperform simpler deep learning models like GRU. Overall, CNN-LSTM provides moderate performance, balancing trend capture with some loss in fine-grained accuracy

1.12 Attention LSTM

The attention mechanism allows the model to dynamically assign importance weights to different timesteps in the input sequence. This enables the network to focus on the most relevant historical observations when generating predictions. Attention-LSTM is used to enhance traditional LSTM models by allowing the network to focus on the most important time steps in the input sequence. In standard LSTM, all past information is treated equally, which can dilute the impact of critical patterns. The attention mechanism assigns different weights to different time steps, helping the model prioritize relevant historical data. This is especially useful in stock price prediction, where certain past events or trends may have a stronger influence on future prices. By focusing on key information, Attention-LSTM can improve prediction accuracy and interpretability. It also helps

overcome the limitations of long sequences, where important signals may otherwise be lost. In this project, it is used to explore whether selective focus on historical data improves forecasting performance.

```
[33]: from tensorflow.keras.models import Model
      from tensorflow.keras.layers import Input, LSTM, Dense, Dropout

      class TemporalAttention(Layer):
          def build(self, input_shape):
              self.W = self.add_weight(
                  name='att_weight',
                  shape=(input_shape[-1], 1),
                  initializer='glorot_uniform',
                  trainable=True
              )
              self.b = self.add_weight(
                  name='att_bias',
                  shape=(1,),
                  initializer='zeros',
                  trainable=True
              )
              super().build(input_shape)

          def call(self, inputs):
              score = tf.tensordot(inputs, self.W, axes=1) + self.b
              weights = tf.nn.softmax(score, axis=1)
              context = tf.reduce_sum(weights * inputs, axis=1)
              return context
```

```
[93]: inputs = Input(shape=(X_train.shape[1],X_train.shape[2]))

      lstm = LSTM(64,return_sequences=True)(inputs)
      lstm = Dropout(0.2)(lstm)

      attention = TemporalAttention()(lstm)
      output = Dense(y_train_all.shape[1])(attention)

      att_model = Model(inputs,output)

      att_model.compile(
          optimizer="adam",
          loss="mse"
      )

      history_att = att_model.fit(
          X_train,
          y_train_all,
```

```
epochs=20,  
batch_size=32,  
validation_split=0.1  
)
```

```
Epoch 1/20  
50/50          5s 33ms/step -  
loss: 0.0195 - val_loss: 0.0164  
Epoch 2/20  
50/50          2s 38ms/step -  
loss: 0.0017 - val_loss: 0.0143  
Epoch 3/20  
50/50          2s 34ms/step -  
loss: 0.0015 - val_loss: 0.0136  
Epoch 4/20  
50/50          2s 24ms/step -  
loss: 0.0013 - val_loss: 0.0114  
Epoch 5/20  
50/50          1s 29ms/step -  
loss: 0.0013 - val_loss: 0.0092  
Epoch 6/20  
50/50          1s 22ms/step -  
loss: 0.0011 - val_loss: 0.0105  
Epoch 7/20  
50/50          1s 24ms/step -  
loss: 0.0010 - val_loss: 0.0109  
Epoch 8/20  
50/50          1s 24ms/step -  
loss: 0.0011 - val_loss: 0.0098  
Epoch 9/20  
50/50          1s 22ms/step -  
loss: 0.0010 - val_loss: 0.0101  
Epoch 10/20  
50/50          1s 24ms/step -  
loss: 0.0010 - val_loss: 0.0119  
Epoch 11/20  
50/50          1s 26ms/step -  
loss: 9.7254e-04 - val_loss: 0.0096  
Epoch 12/20  
50/50          3s 31ms/step -  
loss: 9.5918e-04 - val_loss: 0.0126  
Epoch 13/20  
50/50          2s 42ms/step -  
loss: 9.6825e-04 - val_loss: 0.0113  
Epoch 14/20  
50/50          1s 24ms/step -  
loss: 9.2915e-04 - val_loss: 0.0149  
Epoch 15/20
```

```

50/50          1s 22ms/step -
loss: 9.2940e-04 - val_loss: 0.0088
Epoch 16/20
50/50          2s 35ms/step -
loss: 8.7170e-04 - val_loss: 0.0173
Epoch 17/20
50/50          1s 22ms/step -
loss: 9.2406e-04 - val_loss: 0.0134
Epoch 18/20
50/50          1s 23ms/step -
loss: 9.3800e-04 - val_loss: 0.0099
Epoch 19/20
50/50          1s 22ms/step -
loss: 8.2374e-04 - val_loss: 0.0107
Epoch 20/20
50/50          1s 24ms/step -
loss: 8.2826e-04 - val_loss: 0.0136

```

```

[94]: attention_pred = att_model.predict(X_test)

attention_pred_full = scaler.inverse_transform(attention_pred)

attention_rmse = np.sqrt(mean_squared_error(y_test_actual, attention_pred_full))
mae_attention = mean_absolute_error(y_test_actual, attention_pred_full)
mape_attention = np.mean(np.abs((y_test_actual - attention_pred_full) /
    ↪ y_test_actual)) * 100

print("Attention LSTM RMSE:", attention_rmse)
print("Attention LSTM MAE:", mae_attention)
print("Attention LSTM MAPE:", mape_attention)

```

```

14/14          1s 50ms/step
Attention LSTM RMSE: 29.892844563678683
Attention LSTM MAE: 24.199071712018142
Attention LSTM MAPE: 14.766355268179392

```

```

[95]: plt.plot(history_att.history['loss'],label='Training Loss')
plt.plot(history_att.history['val_loss'],label='Validation Loss')

plt.title("Attention LSTM Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()

plt.show()

```

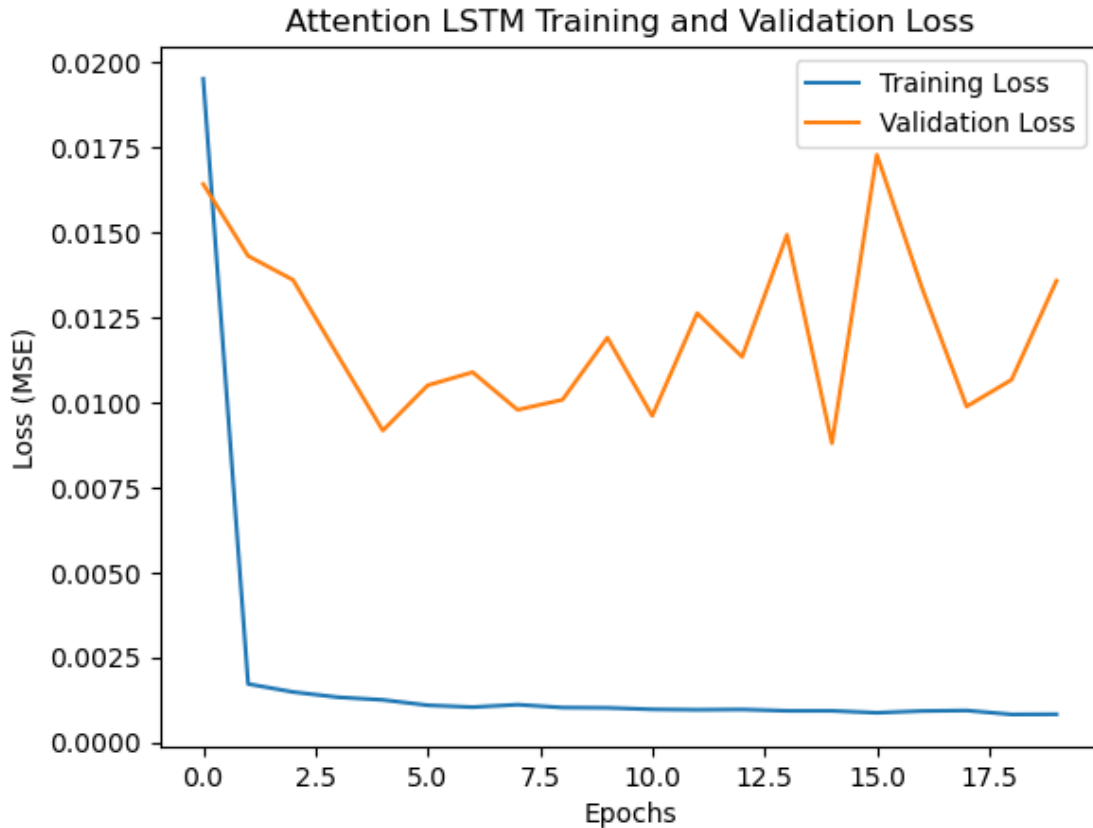


Figure 11: Training and validation loss curves during the model training process

The Attention-LSTM model enhances the traditional LSTM by assigning importance weights to different time steps in the input sequence. After training, the model generates predictions on the test dataset, which are then converted back to the original scale for proper evaluation. The RMSE value (approx 29.89) indicates a relatively higher prediction error compared to other models, suggesting less accurate overall performance. The MAE (approx 24.20) shows that, on average, predictions deviate by around 20 units from the actual stock prices. The MAPE (approx 14.77%) indicates that the model's predictions are approximately 15% off on average, which is higher than most other models used in this project.

Although attention mechanisms are designed to improve performance by focusing on important time steps, in this case, the model may not have effectively learned the most relevant patterns. This could be due to limited data, improper tuning, or the nature of the dataset. Compared to models like GRU and LSTM, the Attention-LSTM performs worse, indicating that added complexity does not always guarantee better results. Overall, the model demonstrates the importance of proper tuning and model selection in time-series forecasting.

```
[46]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0], label="Actual Price")
```

```
plt.plot(attention_pred_full[:,0], label="Attention-LSTM")

plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")
plt.title("Attention LSTM Prediction")
plt.legend()

plt.show()
```

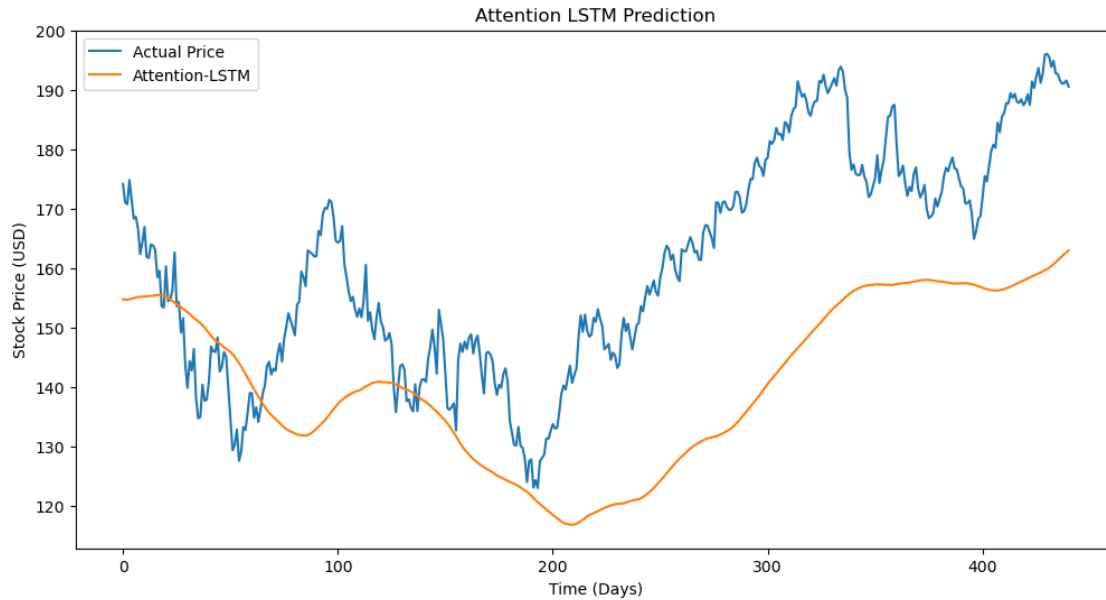


Figure 12: Comparison between the actual stock prices and the prices predicted by the LSTM-Attention model during the testing period

The graph illustrates the comparison between the actual stock prices (blue line) and the predictions generated by the Attention-LSTM model (orange line) over the test period. The predicted values follow the general trend of the stock prices, capturing the overall upward and downward movements. However, the predictions appear significantly smoother and less responsive to short-term fluctuations and sudden price changes. This indicates that the model is focusing more on long-term patterns rather than immediate variations.

There are noticeable gaps between the predicted and actual values, especially during sharp peaks and dips, where the model underestimates volatility. The attention mechanism is expected to highlight important time steps, but in this case, it does not significantly improve prediction accuracy. The model seems to struggle in capturing complex market dynamics and rapid changes in stock prices.

Compared to other models like GRU and LSTM, the Attention-LSTM shows weaker alignment with the actual data. Overall, while the model successfully captures general trends, it lacks precision in tracking detailed price movements, resulting in higher prediction errors.

1.13 Comparison

```
[112]: results = pd.DataFrame({
    "Model": ["LSTM", "GRU", "Random Forest", "XGBoost", "CNN-LSTM", "Attention_LSTM"],
    "RMSE": [rmse_lstm, rmse_gru, rf_rmse, xgb_rmse, cnn_rmse, attention_rmse],
    "MAE": [mae, mae_gru, mae_rf, mae_xgb, mae_cnn, mae_attention],
    "MAPE": [mape, mape_gru, mape_rf, mape_xgb, mape_cnn, mape_attention],
})

results = results.sort_values("RMSE", ascending=True)
results
```

```
[112]:
```

	Model	RMSE	MAE	MAPE
1	GRU	9.956095	7.208282	4.137682
4	CNN-LSTM	15.653963	12.232528	6.673621
0	LSTM	17.778219	14.615277	8.526456
3	XGBoost	18.710316	13.807039	7.997174
5	Attention LSTM	29.892845	24.199072	14.766355
2	Random Forest	39.262932	31.241669	17.555261

The table above presents the performance comparison of multiple predictive models used in this study, including LSTM, GRU, Random Forest, XGBoost, CNN-LSTM, and Attention LSTM. The models are evaluated using Root Mean Squared Error (RMSE), which measures the average difference between predicted and actual stock prices. A lower RMSE value indicates that the model produces predictions that are closer to the real market values. In addition, MAE and MAPE are also used to evaluate and compare the models.

From the results, the GRU model achieved the lowest RMSE value, indicating the best predictive performance among all the tested models. This is followed by CNN-LSTM with a RMSE of 15.65. The LSTM model produced a higher RMSE value of around 17.78, suggesting that its predictions are less accurate compared to the GRU model. Despite being a more complex deep learning model, the Attention LSTM model resulted in almost the highest error, meaning it is not a suitable model for this data.

The traditional machine learning model, Random Forest, performed significantly worse than the deep learning models, producing the highest RMSE value of 39. This suggests that this model was unable to effectively capture the temporal dependencies present in the financial time-series data. XGBoosting got the second lowest error, as it is also based on sequential learning.

Overall, the results demonstrate that recurrent neural network architectures, particularly GRU, are more suitable for stock price forecasting tasks because they are specifically designed to learn sequential patterns in time-series data.

```
[113]: x = np.arange(len(results["Model"]))
width = 0.25

plt.figure(figsize=(10,5))
```

```

plt.bar(x - width, results["RMSE"], width, label="RMSE")
plt.bar(x, results["MAE"], width, label="MAE")
plt.bar(x + width, results["MAPE"], width, label="MAPE")

plt.xticks(x, results["Model"], rotation=45)
plt.title("Model Performance Comparison")
plt.ylabel("Error")
plt.legend()

plt.show()

```

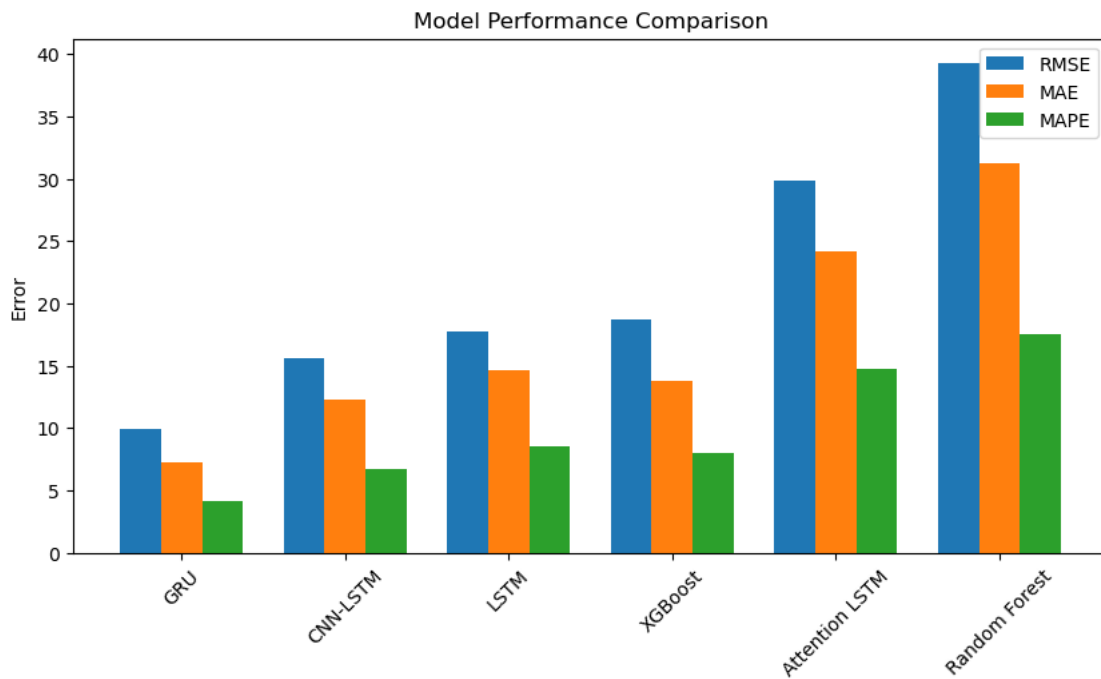


Figure 13: Comparison of RMSE, MAE and MAPE values for different machine learning and deep learning models used for stock price prediction.

The above figure presents a visual comparison of the prediction performance of five different models used in this study: LSTM, GRU, Random Forest, XGBoost, and CNN-LSTM. The models are evaluated using the Root Mean Squared Error (RMSE) metric, where lower RMSE values indicate better prediction accuracy. From the graph, it can be observed that the GRU model achieved the lowest metric values, indicating the highest predictive accuracy among all the models tested.

```

[114]: plt.figure(figsize=(12,6))

plt.plot(y_test_actual[:,0], label="Actual")
plt.plot(predictions[:,0], label="LSTM")
plt.plot(gru_pred[:,0], label="GRU")

```

```
plt.plot(cnn_pred_full[:,0], label="CNN-LSTM")
plt.plot(attention_pred_full[:,0], label="Attention-LSTM")

plt.xlabel("Time (Days)")
plt.ylabel("Stock Price (USD)")
plt.title("Stock Price Prediction: Model Comparison")

plt.legend()
plt.grid(True)

plt.show()
```

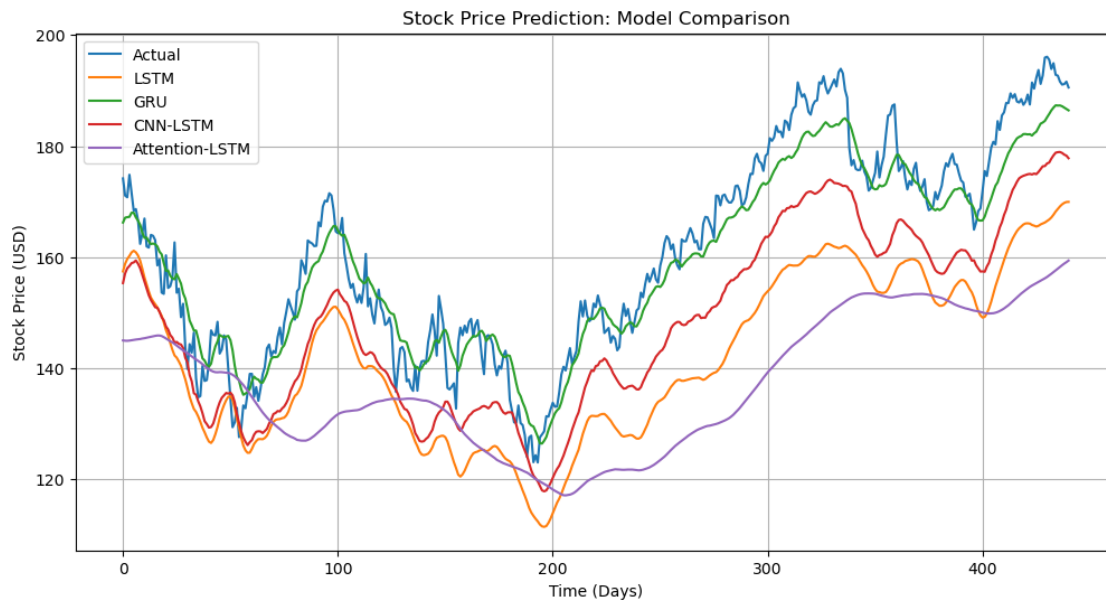


Figure 14: Comparison between the actual stock prices and the prices predicted by all the models during the testing period

The above graph illustrates the comparison between the actual stock prices (blue line) and the predictions generated by the Attention-LSTM model (orange line) over the test period. The predicted values follow the general trend of the stock prices, capturing the overall upward and downward movements. However, the predictions appear significantly smoother and less responsive to short-term fluctuations and sudden price changes. This indicates that the model is focusing more on long-term patterns rather than immediate variations.

There are noticeable gaps between the predicted and actual values, especially during sharp peaks and dips, where the model underestimates volatility. The attention mechanism is expected to highlight important time steps, but in this case, it does not significantly improve prediction accuracy. The model seems to struggle in capturing complex market dynamics and rapid changes in stock prices.

Compared to other models like GRU and LSTM, the Attention-LSTM shows weaker alignment with

the actual data. Overall, while the model successfully captures general trends, it lacks precision in tracking detailed price movements, resulting in higher prediction errors.

```
[115]: last_sequence = X_test[-1]

future_predictions = []

for i in range(5):

    pred = gru_model.predict(last_sequence.reshape(1,60,X.shape[2]))

    future_predictions.append(pred[0][0])

    last_sequence = np.append(last_sequence[1:], pred, axis=0)

print("Next 5 predicted prices:", future_predictions)
```

```
1/1          0s 112ms/step
1/1          0s 133ms/step
1/1          0s 99ms/step
1/1          0s 104ms/step
1/1          0s 115ms/step
Next 5 predicted prices: [np.float32(0.9450556), np.float32(0.9398304),
np.float32(0.93432456), np.float32(0.92914784), np.float32(0.9242954)]
```

This section represents the final stage of the project, where the trained GRU model is used to forecast future stock prices based on the most recent available data. The last sequence of 60 time steps from the test dataset is taken as input, reflecting the sliding window approach used during training. The model then predicts the next value, which is appended to the sequence while removing the oldest value, allowing iterative multi-step forecasting. This process is repeated five times to generate predictions for the next five future time steps.

The predicted values are in normalized form (due to MinMax scaling), indicating relative price levels rather than actual stock prices. These predictions show a slight downward trend, suggesting a short-term decline based on recent patterns learned by the model.

Compared to other models in the project, GRU demonstrated the best performance with the lowest RMSE and MAPE, indicating higher accuracy and better generalization. LSTM and CNN-LSTM captured trends reasonably well but had higher errors, while Attention-LSTM introduced complexity without improving results. Traditional models like Random Forest and XGBoost struggled with temporal dependencies, leading to less accurate forecasts.

Overall, this result highlights that GRU is the most effective model in this project for capturing time-series dynamics and generating reliable future predictions.

1.14 Discussion

The objective of this project was to evaluate the effectiveness of different machine learning and deep learning models in predicting stock prices using historical financial data. Various models including LSTM, GRU, CNN-LSTM, Attention LSTM, Random Forest, and XGBoost were implemented

and compared using evaluation metrics such as RMSE, MAE, and MAPE. The results indicate that deep learning models designed for sequential data are more effective in capturing the temporal dependencies present in financial time-series datasets. Among all models tested, the GRU model achieved the lowest RMSE and MAPE values, indicating superior predictive accuracy.

The visualization of actual versus predicted stock prices further demonstrates the ability of recurrent neural networks to capture long-term patterns in stock price movements. While both CNN-LSTM and GRU models successfully followed the overall trend of stock prices, the GRU model showed better alignment with actual values and produced lower prediction errors. In contrast, traditional machine learning models such as Random Forest performed poorly compared to deep learning approaches. These models are not inherently designed to handle sequential dependencies in time-series data, which likely explains their relatively high error values.

Residual analysis also provided insight into the performance of the predictive models. The residual distribution showed that most prediction errors were centered around zero, indicating that the model predictions are generally unbiased and reasonably accurate. However, some larger errors were observed during periods of high market volatility, which is expected due to the unpredictable nature of financial markets. Overall, the findings suggest that recurrent neural networks, particularly GRU, are well-suited for modeling financial time-series data and can provide meaningful insights for stock price forecasting.

1.15 Conclusion

In this project, several machine learning and deep learning techniques were applied to predict stock prices using historical market data. The study demonstrated that deep learning architectures designed for sequential data, such as LSTM and GRU, outperform traditional machine learning algorithms when dealing with financial time-series forecasting tasks. By utilizing past stock price sequences as input, the models were able to learn complex patterns and dependencies within the data.

The experimental results showed that the GRU model achieved the best performance among all implemented models, producing the lowest RMSE and MAPE values. This indicates that the GRU model was more effective in capturing temporal relationships in stock price movements compared to the other models. Although the LSTM model also performed reasonably well, its prediction error was higher than that of GRU. Meanwhile, the Random Forest produced significantly higher error values, highlighting the limitations of traditional machine learning approaches in time-series prediction tasks.

Overall, this study reinforces the effectiveness of recurrent neural networks for financial forecasting applications. Despite the inherent challenges of stock price prediction, including market volatility and external influences, the models were able to generate meaningful and reasonably accurate predictions. The findings suggest that deep learning-based approaches, particularly GRU, provide a promising direction for future research and real-world financial analytics. However, further improvements can be achieved by incorporating additional features such as technical indicators, macroeconomic variables, and advanced architectures to enhance prediction accuracy.

1.16 Limitations

- 1) Limited Feature Set The models in this project were trained primarily using historical closing prices. While closing prices contain important information about market trends, they do not

capture other influential factors such as trading volume, technical indicators, macroeconomic conditions, or investor sentiment. The absence of these additional features may limit the predictive power of the models.

- 2) **Market Volatility and External Factors** Stock markets are inherently volatile and are influenced by many unpredictable external factors such as economic policies, geopolitical events, global crises, and company-specific news. These external influences were not included in the dataset used for training the models, which may lead to prediction errors during sudden market changes.
- 3) **Limited Dataset Scope** The analysis was conducted on a small set of major technology companies including Apple, Microsoft, Google, Amazon, and Tesla. Although these companies provide valuable insights into market trends, the limited number of stocks may restrict the generalization of the results across other industries and market conditions.
- 4) **Traditional Models Not Designed for Time-Series Data** Some of the models used in the project, such as Random Forest and XGBoost, are not specifically designed for sequential time-series data. As a result, these models were less effective in capturing temporal dependencies in stock prices, which contributed to their relatively higher prediction errors.

1.17 Future Work

- 1) **Incorporating Additional Financial Indicators** Future research could improve model performance by including additional financial indicators such as trading volume, moving averages, RSI (Relative Strength Index), and other technical indicators. These variables may provide deeper insights into market behavior and improve prediction accuracy.
- 2) **Integration of Sentiment and News Data** Stock prices are often influenced by market sentiment and news events. Incorporating sentiment analysis from financial news articles, social media platforms, or earnings reports could provide valuable contextual information for improving prediction models.
- 3) **Using Advanced Deep Learning Architectures** Future studies could explore more advanced architectures such as Transformer-based models, Attention mechanisms, or Temporal Convolutional Networks (TCNs). These models have shown strong performance in sequence modeling tasks and may provide better results for financial time-series forecasting.
- 4) **Expanding the Dataset and Prediction Horizon** The dataset can be expanded to include more companies from different industries and global markets. Additionally, future models could focus on multi-step forecasting, predicting stock prices over longer future periods rather than only short-term predictions.

1.18 Appendix

1.18.1 AI-Statement

We used AI to make minor adjustments to the code in order to improve the analysis alongside checking grammar and language of the report.